MYNDNOVA SOLUTIONS ◆ CRUISESTREAM STUDIOS

Day 1 — AWS + Databricks AI Foundation

Tom Cruise Movie Data on Databricks · CruiseStream Studios

3

Modules

3

Labs

6

Hours

Dr. Yogesh Ghorpade

Solutions Architect — AI & Data ◆ MyndNova Solutions

linkedin.com/in/yogesh-ghorpade

Today's Agenda

1

AI Fundamentals

AI vs GenAI vs Agentic AI
RAG vs Agents
AWS + Databricks Architecture

2

Data Foundation

S3 Storage
Delta Lake & ACID
Medallion: Bronze/Silver/Gold

3

Governance

Unity Catalog
Column Masking
AI-Safe Views

MODULE 1 / 03

Three Types of AI

AI

Artificial Intelligence

Recognizes patterns — spam filters, recommendations, fraud detection

GENAI

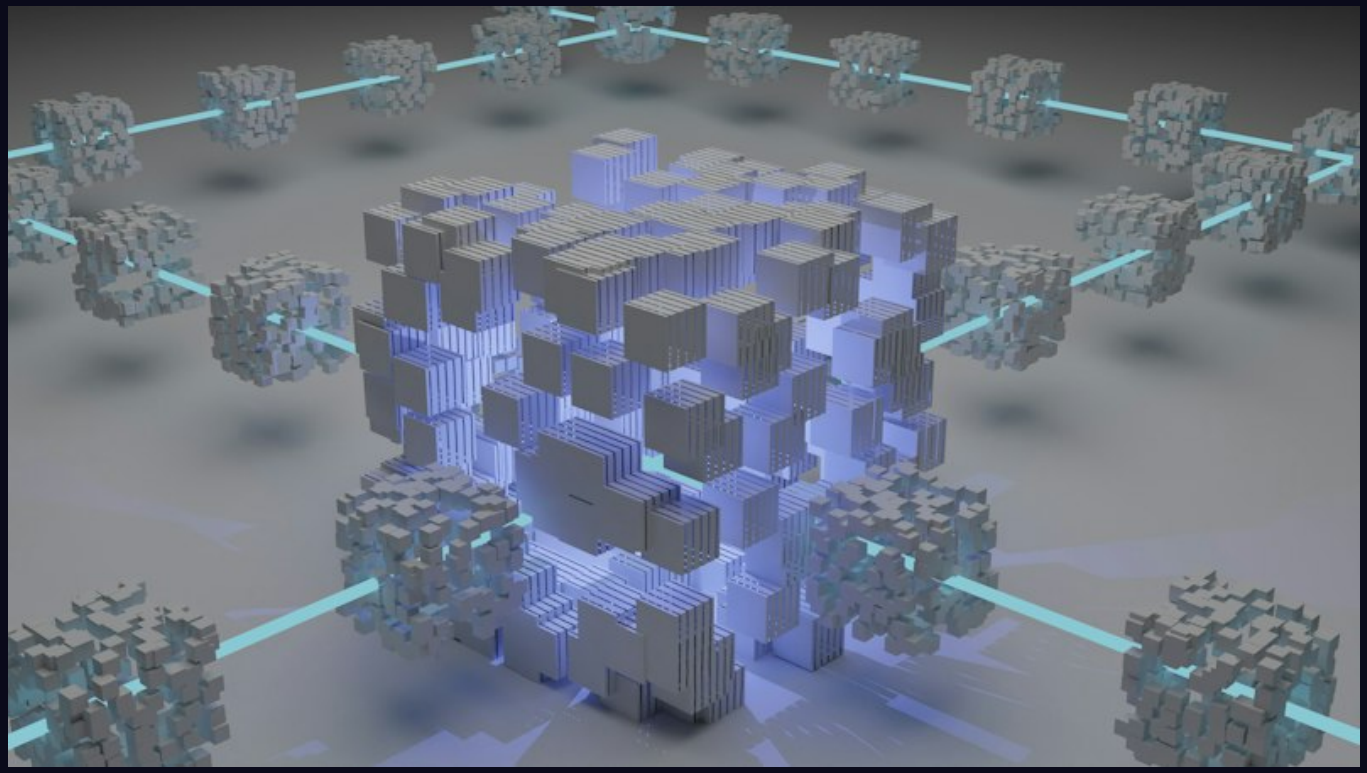
Generative AI

Creates new content — summaries, Q&A, genre classification

AGENT

Agentic AI

Takes actions — SQL queries, plot search, box office calculator



MODULE 1 / 04

RAG vs AI Agents

Key insight: RAG answers questions. Agents *solve problems* by chaining multiple tools autonomously.

MODULE 1 / 05

CruiseStream Architecture

■ **ARCHITECTURE FLOW**Raw S3 → IAM secures access → Databricks processes (Delta + UC) → AI Agents query → User gets answers.

LAB 1 / 06

Hands-On Lab

Lab 1: Design the Architecture

- Draw the end-to-end architecture for CruiseStream Studios
- Identify where raw movie data enters (S3)
- Map IAM roles for secure data access
- Define what Databricks provides: Delta Lake, UC, AI Models
- Plan where AI agents will query and analyze movie data

Deliverable: Architecture diagram S3 → IAM → Databricks → AI Agents.

MODULE 2 / 07

S3 — Raw Data Source

```
-- S3 bucket structure
s3://cruisestream-raw-data/
 ■-- movies/tom_cruise_movies.csv
 ■-- plots/movie_plot_summaries.txt

-- IAM role grants Databricks
-- read access to S3
-- No credentials in code!
```

■ **KEY LOGIC**IAM handles authentication — no API keys in code. Databricks reads S3 directly via instance profile role.



Use Case: 12 Tom Cruise movies stored as CSV in S3. Databricks reads via IAM role — zero credentials exposed.

MODULE 2 / 08

Delta Lake — Smart Storage

ACID

All-or-nothing writes

Time Travel

Query any past version

Schema Enforcement

Prevents bad data

```
CREATE TABLE IF NOT EXISTS ...bronze_movies_raw (
   title STRING, release_year INT, genre STRING,
   imdb_rating DOUBLE, box_office_millions DOUBLE, summary STRING
) USING DELTA;

SELECT * FROM ...bronze_movies_raw VERSION AS OF 1;
```

■ **KEY LOGIC** `USING DELTA` enables ACID transactions + time travel. `VERSION AS OF 1` queries data at version 1 — useful for auditing.

MODULE 2 / 09

Medallion Architecture

Like cooking: raw ingredients (Bronze) → prepped & enriched (Silver) → finished dish (Gold)

Bronze

Raw CSV from S3, loaded as-is into Delta. No transformations — audit trail preserved.

Silver

CASE WHEN enrichment: decade, rating_category, is_blockbuster flag.

Gold

GROUP BY genre/decade: avg_rating, total_box_office — ready for AI agents.

MODULE 2 / 10

Bronze Layer Implementation

```
INSERT INTO ...bronze_movies_raw
SELECT title, release_year, genre, imdb_rating,
     box_office_millions, summary
FROM lakemeter_catalog.labuser3.tom_cruise_movies;

SELECT COUNT(*) AS total_movies FROM ...bronze_movies_raw;
-- Result: 12

-- Sample: Top Gun | 1986 | Action | 6.9 | 357.0
--         Rain Man | 1988 | Drama | 8.0 | 354.8
```

■ **KEY LOGIC**Bronze copies raw data *as-is* from source. No transformations — just safe Delta storage. Preserves original data for audit trails.

MODULE 2 / 11

Silver Layer Implementation

```
CREATE OR REPLACE TABLE ...silver_movies_clean AS
SELECT title, release_year, genre, imdb_rating,
     box_office_millions, summary,
   CASE WHEN release_year BETWEEN 1980 AND 1989 THEN '1980s'
       WHEN release_year >= 2020 THEN '2020s' END AS decade,
   CASE WHEN imdb_rating >= 8.0 THEN 'Excellent'
       WHEN imdb_rating >= 7.0 THEN 'Good' ELSE 'Average' END AS rating_category,
   box_office_millions >= 500 AS is_blockbuster
FROM ...bronze_movies_raw;
```

■ **KEY LOGIC**Three `CASE WHEN` blocks: **decade** (time bucketing), **rating_category** (quality tier), **is_blockbuster** (boolean flag). Enriches raw data for AI.

MODULE 2 / 12

Gold Layer Implementation

```
CREATE OR REPLACE TABLE ...gold_movie_summary AS
SELECT genre, decade,
   COUNT(*) AS movie_count,
   ROUND(AVG(imdb_rating), 2) AS avg_rating,
   SUM(box_office_millions) AS total_box_office,
   MAX(box_office_millions) AS highest_grossing
FROM ...silver_movies_clean
GROUP BY genre, decade ORDER BY genre, decade;
```

■ **KEY LOGIC**`GROUP BY genre, decade` creates a pivot table. `AVG`, `SUM`, `MAX` provide business-ready metrics. AI agents query Gold for analytics.

Scenario: Executives ask "Which genre has highest avg box office?" Gold table: Action $457M in 1990s.

MODULE 3 / 13

Unity Catalog — AI Governance

Column Masking

Hides box office $ — shows categories

Column Tagging

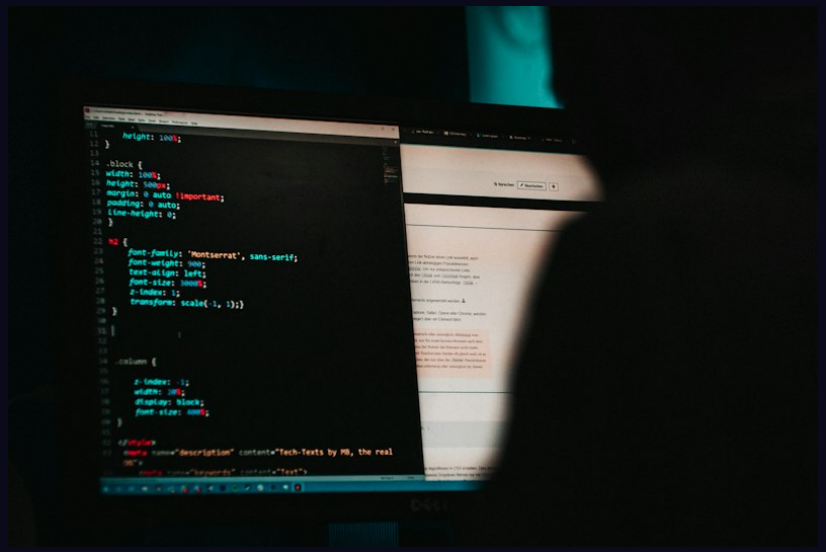
Tags financial + content columns

Descriptions

AI-readable column docs

AI-Safe View

Masked view for agents



MODULE 3 / 14

Column Masking Implementation

```
CREATE FUNCTION mask_box_office(val DOUBLE)
RETURNS STRING
RETURN CASE WHEN val >= 500 THEN 'Blockbuster'
           WHEN val >= 200 THEN 'Hit'
           WHEN val >= 100 THEN 'Moderate' ELSE 'Low' END;

ALTER TABLE ...silver_movies_clean
   ALTER COLUMN box_office_millions
   SET MASK mask_box_office;

ALTER TABLE ...silver_movies_clean
   ALTER COLUMN box_office_millions SET TAGS ('financial'='true');
COMMENT ON COLUMN ...silver_movies_clean.title
   IS 'Official movie title (e.g., Top Gun, Rain Man)';
```

■ **KEY LOGIC**`CREATE FUNCTION` defines masking. `SET MASK` applies it — users see "Blockbuster" not $591M. `SET TAGS` marks sensitivity. `COMMENT` gives AI schema understanding.

MODULE 3 / 15

AI-Safe View Implementation

```
CREATE OR REPLACE VIEW ...v_movies_ai_safe AS
SELECT title, release_year, genre, imdb_rating,
   mask_box_office(box_office_millions) AS box_office_category,
   box_office_tier, is_blockbuster, decade, rating_category, summary
FROM ...silver_movies_clean;

-- AI sees: Top Gun | 1986 | Action | 6.9 | Hit | ...
-- (NOT the raw $357.0 million)
```

■ **KEY LOGIC**The view calls `mask_box_office()` inline — replaces raw $ with category. AI agents query `v_movies_ai_safe` and can never expose financial data.

Scenario: AI asks "What's Top Gun's revenue?" Sees "Hit" not $357M. Compliance: AI can't leak financial data.

LAB 3 / 16

Hands-On Lab

Lab 3: Apply Governance

- Add column descriptions to silver_movies_clean
- Tag box_office_millions as "financial", summary as "content"
- Create masked view v_movies_ai_safe
- Verify governance via SQL inspection
- Confirm AI agents only see safe, masked data

Result: All 12 movies documented, tagged, masked. AI sees categories, not raw financial numbers.

MODULE 3 / 17

Real-World Use Cases

Streaming Recommendations

Gold table shows Top Gun + M:I are both Action/high box office → AI recommends M:I to Top Gun viewers.

Financial Compliance

Masked view shows "Blockbuster" not $591M — enough for audit without exposing revenue.

Data Discovery

New AI agent reads descriptions: "title=movie name, is_blockbuster=high-revenue flag." Instant schema understanding.

Trend Analysis

"Best decade for Tom Cruise?" Gold table: 1980s had Rain Man (8.0), 2010s had Edge of Tomorrow (7.9).

Day 1 Recap

Module 1

AI Fundamentals

AI → GenAI → Agents
RAG vs Agent workflow
AWS S3 + IAM + Databricks

Module 2

Data Foundation

Delta Lake ACID + time travel
Bronze → Silver → Gold
CASE WHEN + GROUP BY

Module 3

Governance

Masking function + SET MASK
Tags + descriptions
AI-safe masked view

Tomorrow: GenAI to summarize plots, classify genres, build Vector Search, and create a RAG pipeline with anti-hallucination.

Dr. Yogesh Ghorpade
MyndNova Solutions ◆ Databricks Certified
linkedin.com/in/yogesh-ghorpade